FLYRANK INTERNSHIP · BACKEND TRACK · WEEK 5 · ASSIGNMENT A9

The polite scraper

Download three catalogue pages from a practice sandbox, visit all 60 book pages, turn messy HTML into clean, checked JSON — politely, without crashing on a broken page, and with an honest report at the end of every run.

~5–6 h across 7 stages | Stretch +2 h | Bonus AI stage +1 h | JavaScript or Python | $0 · no credit card

PAIRED LIVE EVENT
Workshop — Scraping in practice: from messy HTML to clean data

YOU WILL PRACTISE
Target classification · polite fetching · HTML extraction · normalization · schema validation · surviving failures · a run report

SUBMISSION
Public GitHub repo, 7+ commits, README a stranger can run in 5 minutes

How to read this document: new words are shown in **bold** the first time they appear — every one of them is explained in the Glossary at the end. Work the stages **in order**; each ends with a checkpoint that proves it works. The only site you will touch is a public practice sandbox that exists exactly for this. If you only finish Stage 3, submit anyway.

Contents

1 · Goal & purpose

Goal: build a small, polite **scraping pipeline**: it downloads the first three catalogue pages of Books to Scrape, visits all 60 book pages, turns messy HTML into clean, checked JSON records, survives a broken page without crashing, and ends every run with a short report of what happened.

Almost every AI system starts with the same question: where does the data come from? A model is good at reasoning about text — but it should not have to re-read a web page every day just to find a price. Something has to collect that data first, reliably and respectfully. That something is a scraper, and this week you build one.

Three professional habits start here, and they follow you for the rest of the track:

1. **Check before you collect.** You classify the target before you write any code.
2. **Be a polite guest.** You say who you are, you go slowly, and you never hammer a site.
3. **Trust nothing you scraped.** A web page is **untrusted input** — it must be checked before it is stored.

This looks bigger than your first assignments, but every stage is small: build one step, look at its output with your own eyes, move on. FlyRank runs this exact shape for audits and content pipelines: fetch → extract → normalize → validate → store → report.

> **If you only finish Stage 3, submit anyway** — a working half is worth more than a broken whole.

2 · The big idea in 60 seconds

A scraper is not "open a page and copy the text." It is a chain of small steps, and each step can prove it did its job:

Step	The question it answers	The proof
Classify	May I automate this site?	a short note in your README
Fetch	Did the page really arrive?	a saved HTML file + status `200`
Extract	Which parts of the page do I need?	raw text fields
Normalize	How does `"£51.77"` become a number?	clean values, absolute URLs
Validate	Is every record safe to store?	a schema check; bad records set aside
Store	Can another program use this?	`books.json`
Report	Did the run actually work?	a few honest numbers

That table **is** the assignment — roughly one stage per row. And AI comes *after* this pipeline, not instead of it: **scrape deterministically, reason semantically, validate both.**

3 · Tools — pick ONE lane

Both lanes build the same records and pass the same checkpoints. Pick one lane and stay in it.

	JavaScript lane	Python lane
Language	Node.js 20+ (free, nodejs.org)	Python 3.10+ (free, python.org)
HTTP request	Built-in `fetch`	Requests
HTML parser	Cheerio	Beautiful Soup
Schema validator	Zod	Pydantic
Output	Built-in file system → JSON	Built-in `json` module → JSON
Target	Books to Scrape — a sandbox built for practice	
Publishing	Git + a free GitHub account	

> Not sure? Stay with the language you used earlier in the track. No database, paid proxy, cloud account, or credit card is needed — and the only site you will touch is a public practice sandbox that exists exactly for this.

4 · The task — seven stages (+ one bonus)

Work the stages **in order**. Each one ends with a checkpoint you can run in under a minute, and a commit — that gives you 7+ meaningful commits, honestly earned.

0 Check before you collect

~30 min

Before entering a building, check whether the door is meant for you.

1. Create a `scraper/` folder in your internship repo, with a `README.md`, a `.gitignore`, and one entry file (`src/index.js` or `src/main.py`).
2. Open toscrape.com and read what the site says about itself. Confirm with your own eyes that Books to Scrape is a **sandbox** — a site built so people can practise scraping on it. That sentence on their page is your permission, and a site like this is the *only* kind this assignment touches.
3. Request `https://books.toscrape.com/robots.txt` once and write down what happened. `robots.txt` is a site's note to robots about where they are welcome. If the file is missing, write "no robots file found" — a missing file is not permission, it is just a missing file.
4. Add a short "Target classification" section to your README: which site, why, how much (the first 3 catalogue pages only), what data you collect, and one sentence on why that is appropriate here.

> **CHECKPOINT** — your README names the target, the three-page scope, the robots result, and this sentence: "I will not reuse this code on another site without checking its rules and terms first."

Commit: `Stage 0: classify scraping target`

1 Fetch once, cache once

~45 min

Before sorting the mail, prove that it arrived.

1. Download the first catalogue page. Send an honest **user-agent** that names you, such as `FlyRankInternship-A9/1.0 (+link-to-your-repo)`. This is how a polite robot introduces itself — a site owner who sees it in their logs can find out who you are.
2. Set a **timeout**. A request must give up after a few seconds — never wait forever.
3. Check the status code before doing anything else. Only `200` means "here is your page." Anything else is a failed fetch, not HTML to parse.
4. Save the HTML to `cache/catalogue-page-1.html` — this saved copy is your **cache**. From now on, while developing, read the saved copy instead of asking the site again. You will restart your script fifty times tonight; the site should feel it once.

> **CHECKPOINT** — run the script twice. The first run prints `FETCH` and creates the file; the second prints `CACHE HIT`. Both report the response size, and neither dumps the whole HTML into your terminal.

Commit: `Stage 1: fetch and cache HTML`

2 Find all three pages

~45 min

A crawler finds the shelves; an extractor reads the labels.

1. Parse the saved page with Cheerio or Beautiful Soup.
2. Collect the link to every book on page 1. These links are **relative URLs** (they look like `../book-name/index.html`). Turn each one into an **absolute URL** using your language's URL tools (`new URL(href, pageUrl)` in JS, `urljoin(page_url, href)` in Python) — never by gluing strings together.
3. Follow the catalogue's own "next" link to page 2, then page 3, then stop. Let the site tell you what the pages are — do not hardcode sixty book links.
4. Wait at least half a second between real requests to the site. Cached pages need no delay — they never leave your computer.
5. Remove duplicate links before the next stage.

> **CHECKPOINT** — the script prints `catalogue_pages=3`, `discovered=60`, `unique_urls=60` — and a second run reports the same numbers, mostly from cache.

Commit: `Stage 2: discover three catalogue pages`

3 Extract the raw records

~1 h 15

Now open each book — and keep the receipt.

For every book page, collect this raw record:

```
{
  "title": "A Light in the Attic",
  "product_url": "https://books.toscrape.com/catalogue/a-light-in-the-attic_1000/
index.html",
  "price_text": "£51.77",
  "availability_text": "In stock (22 available)",
  "rating_text": "Three",
  "description": "...",
  "source_page": "https://books.toscrape.com/catalogue/page-1.html",
  "fetched_at": "2026-08-06T10:00:00Z"
}
```

1. Fetch and cache each detail page with the same politeness as Stage 1: user-agent, timeout, status check, delay.
2. Aim your selectors at the product area of the page, not the whole document. "The first thing that looks like a price" works today and betrays you the day the page grows a second price.
3. Some books have no description. Store `null` — never invent text that was not on the page.
4. Keep `source_page` and `fetched_at` on every record. That is **provenance** — the receipt showing where and when a fact came from. When a value looks wrong three weeks from now, the receipt is how you find out what happened.

> **CHECKPOINT** — print one complete raw record and the summary `detail_pages=60`. The record shows all eight keys, even when an optional value is `null`.

Commit: `Stage 3: extract book details`

4 Clean it, check it, store it

~1 h

Raw strings are ingredients. The schema is the recipe.

1. Turn `price_text` (`"£51.77"`) into `price_gbp` (`51.77`) — a real number a program can sort and compare. Keep the original text too: the raw value and the clean value live side by side.
2. Use the absolute `product_url` as each record's identity — its **canonical URL**. If the same book appears twice, it counts once.
3. Write down the shape of a finished record as a schema, with Zod or Pydantic: which fields are required, what type each one is, `description` optional.
4. Validate every record against the schema **before** storing it. A record that fails goes to `errors.json` together with the reason — it never sneaks into `books.json`.
5. Write the good records to `output/books.json`. Running the scraper twice must produce the same 60 records — not 120. That property is called **idempotency**, and it is what makes re-running a failed job safe.

> **CHECKPOINT** — `books.json` has exactly 60 records, every `price_gbp` is a number, every URL starts with `https://` — and after a second run it is still exactly 60.

Commit: `Stage 4: validate normalized records`

5 One bad page must not kill the run

~45 min

A good job finishes its work, then tells you what happened.

1. Handle each page separately, so one broken page is logged and skipped. Fifty-nine good records must survive one bad one.
2. If a request fails with a timeout or a server error (`5xx`), wait a moment and try once more. Do **not** retry a `404` (the page does not exist — asking again will not create it) or a `403` (the site said no — asking again is how a polite robot becomes a pest).
3. End every run by writing `output/run-report.json` with a few honest numbers: start time, duration, pages fetched, cache hits, valid records, invalid records, failed pages. A scraper that reports nothing can fail silently for weeks — the report is how you notice.
4. Prove it works: add one made-up book URL to your list on purpose and run. Break things on your side only — never test failure by hammering the real site.

> **CHECKPOINT** — with one fake URL in the list, the run still finishes, `books.json` still has the 60 good records, and `run-report.json` shows `failed_pages: 1`.

Commit: `Stage 5: survive failures, report the run`

Next week builds on exactly this. Assignment A16 adds the production version of these ideas — full retry rules with backoff and `Retry-After`, structured logs, and finding the hidden data layers behind pages. What you build here is the foundation it stands on, so don't gold-plate Stage 5: simple and working is the goal.

6 Publish the evidence

~45 min · required

Your code is finished when a stranger can run it and trust the result.

1. Push the project to a public GitHub repo. Commit your code and one sample output — not hundreds of cached HTML files (add `cache/` to `.gitignore`).
2. Finish the README: your target classification from Stage 0, one copy-pasteable run command, your lane and how to install it, the record schema, the politeness rules you follow (user-agent, delay, timeout, cache), and one honest limitation.
3. Paste one real `run-report.json` into the README as proof, and add one sentence on why this assignment needed no browser: the data is already in the HTML the server sends, so a browser would only add cost.
4. Add a short ethics note in your own words: use an official API when one exists; never bypass logins, paywalls, or blocks; collect only what you need.

> **CHECKPOINT** — a stranger could clone the repo, run one documented command, and get `books.json` plus `run-report.json` in under 5 minutes. `git log --oneline` shows 7+ meaningful stage commits.

Commit: `Stage 6: publish scraper evidence` — then push everything.

★ Make it yours — optional extras

optional · as many or as few as you like

None of these are required. Pick any or none.

- **CSV export:** produce `books.csv` from the validated records, and note which values had to be flattened. *(Sneaks in: format conversion.)*
- **Changed since last run:** hash each record and report how many are new, changed, unchanged, or gone. *(Sneaks in: change detection.)*
- **Tiny dashboard:** one local HTML page showing record count, price range, failures, and when the data was last fresh. *(Sneaks in: observability.)*
- **Selector fixtures:** save two small HTML files and prove your parser handles a missing description and extra whitespace. *(Sneaks in: **fixtures** — testing without the network.)*
- **Retry like a pro:** upgrade Stage 5's single retry into real retry rules — **exponential backoff** with a little randomness, respect the **Retry-After** header, and write **structured logs** with URL, status, and attempt number. *(Next week's assignment teaches this properly — building it now is a head start, not a requirement.)*

Commit (if you build any): `Extras: <what you added>`

5 · Bonus stage — the AI rematch

B The AI rematch

~1 h · optional — and the most fun

You built the pipeline by hand. Now ask an AI to build the same thing — and review it like an engineer.

1. **Write the prompt yourself. This is the real exercise.** Without looking at this page, describe what you built: the target and its three-page scope, the eight raw fields, the clean schema, the caching, the delay, the user-agent, the timeout, the validation rule, the no-duplicates rule, what happens on a broken page, the run report, and your language.
2. **Generate in quarantine.** The AI's version goes in `ai-version/` or a separate branch. Your hand-built version stays untouched.
3. **Run it** against your own checkpoints. Does it collect all 60? Does a rerun duplicate them? Does one bad page crash it?
4. **Diff it.** Write an "AI vs me" README section answering three questions: What did the AI do better — and do you understand that code? What did it get wrong or silently skip? What did *your prompt* forget to say?
5. **One rematch.** Improve the prompt, regenerate once, and note what changed.

> An AI's output is exactly as good as your specification — and you could only judge it because you built the thing yourself first. **Both halves of that sentence are your career from now on.**

> `CHECKPOINT` — the README has your full prompt, checkpoint results for both versions, and at least three concrete differences.

Commit: `Bonus: AI vs me` (the AI code stays isolated).

6 · Requirements

Done = every box ticked. Each item is checkable in under a minute.

- [ ] One documented command processes exactly the first **3 catalogue pages** and discovers **60 unique book URLs**.
- [ ] Every detail page produces the eight raw fields, plus a numeric `price_gbp`.
- [ ] Records are **schema-validated** before storage; failing records land in `errors.json` with a reason.
- [ ] `output/books.json` holds exactly **60 unique records** — after the first run *and* after a rerun.
- [ ] Every real request sends an identifying user-agent, has a timeout, waits at least 500 ms between requests, and checks the status code; development reads from the cache.
- [ ] The README documents the target classification and the robots check result.
- [ ] One deliberately broken URL is logged and skipped: the run finishes and the good records survive.
- [ ] `output/run-report.json` reports counts, failures, cache hits, and duration.
- [ ] Public GitHub repo with 7+ meaningful commits and a README a stranger can run in under 5 minutes.

Stretch (optional)

- [ ] **Browser cost comparison:** fetch one page of quotes.toscrape.com/js with a plain HTTP request (view the source first — the quotes are not in it), then with Playwright. README line: the time and memory of both, and why the core assignment needed no browser.

- [ ] **Parser tests:** at least five unit tests — price normalization, relative→absolute URLs, missing description, duplicate URLs, one malformed fixture.
- [ ] **Background execution:** if you completed A7, run each page as a queued job with a concurrency cap and idempotent writes.
- [ ] **AI enrichment, locally:** use a local model through Ollama to add a category and a short summary from the validated descriptions. Force the output through a schema, and keep the model's opinion separate from the scraped facts.
- [ ] **The AI rematch** above also counts as a stretch item.

> The deeper version of scraping — reverse-engineering the JSON endpoints that hide behind modern pages — is exactly **next week's assignment, A16**. Everything you build here carries straight into it.

7 · Done means

- A clean run produces 60 validated, unique records and a report that proves how they were collected.
- A rerun produces the same 60 records — no duplicates — and mostly reads from the cache.
- One broken page is skipped and reported; it never takes the run down.
- A public repo a stranger can run in five minutes, with the target classification, the record schema, and the politeness rules written down.

8 · Critical resources

Everything is collected in the vault file **W5 — Scraping, Integrations & Webhooks resources**, leveled as usual (🟢 start here · 🟡 when comfortable · 🟠 deep dive). The shortest path per stage:

Stage	Read this, and only this
Stage 0	The ToScrape sandbox page + Robots Exclusion Protocol, RFC 9309. Read these before writing any request code.
Stages 1–3 JS	Cheerio — loading documents and selecting elements. The W5 resources file also has a Books to Scrape build-along.
Stages 1–3 PY	Beautiful Soup quick start + Real Python's scraper build-along.
Stage 4	Zod or Pydantic — define the record once, validate every page against it.
Stage 5 & the retry extra	HTTP 429 + Retry-After.
Stretch	Playwright — getting started + Ollama — structured outputs for the local, schema-forced enrichment (no account, no card).
Publishing	The Git & GitHub guide in the vault's tooling resources.

All listed tools and reading are free. The core assignment needs no paid API, proxy, cloud service, or credit card.

9 · Glossary

Plain-language definitions of every **bold** technical term above. Read them in any order.

Word	What it means
Scraping pipeline	A repeatable chain of steps that gets web data, extracts fields, cleans them, checks them, and stores the result.
Untrusted input	Data your program did not create and must check before using. Web pages can be missing, malformed, or changed without notice.
Sandbox	A practice environment built for safe experimentation. Books to Scrape exists so people can learn scraping on it.
User-agent	A request header naming the software that makes the request. A polite scraper uses a clear, honest value with a contact link.
Timeout	A maximum wait. When it is reached, the request stops instead of hanging forever.
Cache	A saved copy used again later, so development does not send the same request to the site over and over.
Relative URL	A partial address such as `../book/index.html`. It needs a base address before it can be requested.
Absolute URL	A complete address including `https://` and the site name, such as `https://books.toscrape.com/catalogue/...`.
Provenance	The record of where and when a fact came from — its source URL and fetch time. Kept on every record, never overwritten.
Schema validator	A library that checks whether data has the required fields and the right types. Zod and Pydantic are schema validators.
Canonical URL	The one address chosen as a record's stable identity, even if several addresses could reach the same page.
Idempotency	The property that running the same job twice gives the same result. A rerun updates records — it never duplicates them.
Exponential backoff	Waiting longer after each failed attempt — 1s, then 2s, then 4s — usually with a little randomness added.
Retry-After	A response header where the server says exactly how long to wait before trying again. Obey it instead of guessing.
Structured logs	Log lines with named fields (URL, status, attempt) so both people and programs can search them.
Fixture	A small saved file used to test code again and again without calling a live website.

FlyRank Internship · Backend Development Track · Week 5 · Assignment A9 — The polite scraper. The only target is a public practice sandbox; all linked resources are free with no credit card required.